Sliding Window Templates

Three variants — pick by what the problem asks for.
`i` = left boundary (inclusive), `j` = right boundary (loop variable, inclusive).
Window = `[i, j]`, size = `j - i + 1`.

The Three Templates

```
// FIXED window of size k — shrink exactly once when full
// MENTAL MODEL: a constant-width window slides one step at a time; add the new cell, drop the old one.
// WHEN: "subarray/substring of size k"
int i = 0;
for (int j = 0; j < n; j++) {
    // 1. add nums[j] to state
    if (j - i + 1 == k) {
        // 2. record (window is exactly size k)
        // 3. remove nums[i] from state
        i++;
    }
}

// MAX variable window — find LONGEST valid window
// MENTAL MODEL: grow greedily; shrink only enough to restore validity, then the window is the best ending here.
// WHEN: "longest" + "at most k ..."
int i = 0, result = 0;
for (int j = 0; j < n; j++) {
    // 1. add nums[j] to state
    while (/* window exceeds constraint */) {  // shrink WHILE window exceeds constraint
        // remove nums[i] from state
        i++;
    }
    result = Math.max(result, j - i + 1);   // record after window is valid
}

// MIN variable window — find SHORTEST valid window
// MENTAL MODEL: grow until valid, then shrink aggressively while still valid to find the tightest fit.
// WHEN: "shortest/minimum" + "sum >= target" / "contains all of t"
int i = 0, result = Integer.MAX_VALUE;
for (int j = 0; j < n; j++) {
    // 1. add nums[j] to state
    while (/* window VALID */) {                // record then shrink WHILE window still valid
        result = Math.min(result, j - i + 1);  // record while valid
        // remove nums[i] from state
        i++;                                    // then try to shrink
    }
}
```

Quick Reference Table

#	Name	Description	Problem phrase trigger	Type	State	Shrink / Record condition
643	Max Average Subarray I	Max average of subarray of size k	"subarray of size k"	Fixed	sum	shrink when `j-i+1 == k`
567	Permutation in String	Does s2 contain any permutation of s1?	"substring of size len(s1)" / "permutation"	Fixed	freq + required count	shrink when `j-i+1 == len(s1)`
219	Contains Duplicate II	Any duplicate indices ≤ k apart	"within k indices"	Fixed	HashSet	shrink when `j-i+1 > k`
239	Sliding Window Maximum	Max element in every window of size k	"max of every window of size k"	Fixed	monotone deque	shrink when front out of `[i,j]`
1004	Max Consecutive Ones III	Longest subarray with at most k zeros	"longest" + "at most k zeros"	Max	zero count	shrink while `zeros > k`
3	Longest No-Repeat Substring	Longest substring without duplicates	"longest" + "no repeats"	Max	freq array	shrink while `freq[j] > 1`
424	Longest Char Replacement	Longest with at most k replacements	"longest" + "at most k replacements"	Max	freq + maxFreq	shrink while `size - maxFreq > k`
209	Min Size Subarray Sum	Shortest subarray with sum ≥ target	"shortest" + "sum >= target"	Min	sum	record + shrink while `sum >= target`
76	Minimum Window Substring	Shortest substring containing all of t	"shortest substring containing" all of t	Min (freq)	freq + required count	record + shrink while `required == 0`
159	Longest Substring with At Most 2 Distinct	Max length substring with ≤2 distinct chars	"longest" + "at most 2 distinct"	Max	freq map (size ≤ 2)	shrink while `freq.size() > 2`
340	Longest Substring with At Most K Distinct	Max length substring with ≤k distinct chars	"longest" + "at most k distinct"	Max	freq map (size ≤ k)	shrink while `freq.size() > k`
438	Find All Anagrams in a String	Find all start indices of p's anagrams in s	"all anagrams" / "all windows of size len(p)"	Fixed	freq compare `int[26]`	shrink when `j-i == len(p)`
1343	Number of Subarrays of Size K and Avg ≥ Threshold	Count fixed-size windows with average ≥ threshold	"subarrays of size k" + "average >= threshold"	Fixed	sum (compare `sum >= k*threshold`)	slide when window size `== k`

Frequency Map Trick (used by 567 and 76)

Both use a `need[]` array and a `required` counter to avoid scanning the map each step.

```
// Expanding j — add s.charAt(j):
if (need[s.charAt(j)]-- > 0) required--;   // was needed → satisfied one more

// Shrinking i — remove s.charAt(i):
if (need[s.charAt(i)]++ >= 0) required++;  // was 0 (used up) → now short one
i++;
```

`need[c]` is positive when still needed, zero or negative when excess.
`required` reaches 0 exactly when all characters of t are covered.

Fixed Window — Sum

#643 Maximum Average Subarray I

Description: Find the contiguous subarray of length k with the maximum average value.

Intuition: Max average of fixed length = max sum; slide a width-k window and track the best running sum.

```
class Solution {
    public double findMaxAverage(int[] nums, int k) {
        int i = 0, sum = 0, maxSum = Integer.MIN_VALUE;
        for (int j = 0; j < nums.length; j++) {
            sum += nums[j];
            if (j - i + 1 == k) {
                maxSum = Math.max(maxSum, sum);
                sum -= nums[i++];
            }
        }
        return (double) maxSum / k;
    }
}
```

Time O(n) | **Space** O(1)

Fixed Window — Frequency Map

#567 Permutation in String

Description: Return true if any permutation of s1 appears as a substring of s2.
Key: fixed window of size `s1.length()`. Use `required` counter — no need to compare maps.

Intuition: A permutation is just a fixed-length window whose letter counts match s1, so slide and check the counts.

```
class Solution {
    public boolean checkInclusion(String s1, String s2) {
        int[] need = new int[26];
        for (char c : s1.toCharArray()) need[c - 'a']++;
        int required = s1.length(), i = 0;
        for (int j = 0; j < s2.length(); j++) {
            if (need[s2.charAt(j) - 'a']-- > 0) required--;   // expand
            if (j - i + 1 == s1.length()) {
                if (required == 0) return true;
                if (need[s2.charAt(i) - 'a']++ >= 0) required++;  // shrink
                i++;
            }
        }
        return false;
    }
}
```

Time O(n) | **Space** O(1)

Fixed Window — HashSet

#219 Contains Duplicate II

Description: Return true if any two equal values are at most k indices apart.
Key: fixed window of size k, maintained as a HashSet.

Intuition: Keep only the last k values in a set; a failed insert means a duplicate within k indices.

```
class Solution {
    public boolean containsNearbyDuplicate(int[] nums, int k) {
        Set<Integer> window = new HashSet<>();
        int i = 0;
        for (int j = 0; j < nums.length; j++) {
            if (!window.add(nums[j])) return true;   // duplicate found in window
            if (j - i + 1 > k) window.remove(nums[i++]);
        }
        return false;
    }
}
```

Time O(n) | **Space** O(k)

Fixed Window — Monotone Deque

#239 Sliding Window Maximum

Description: Return the maximum of every contiguous subarray of size k.
Key: monotone deque stores indices in decreasing value order. Front is always the max of current window.

Intuition: A smaller value with a larger one still in the window can never be the max again, so discard it.

```
class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        int[] result = new int[nums.length - k + 1];
        Deque<Integer> queue = new ArrayDeque<>();   // indices, values decreasing front→back
        int i = 0;
        for (int j = 0; j < nums.length; j++) {
            while (!queue.isEmpty() && queue.peekFirst() < i) queue.pollFirst();       // evict out-of-window
            while (!queue.isEmpty() && nums[queue.peekLast()] <= nums[j]) queue.pollLast(); // evict smaller
            queue.offerLast(j);
            if (j - i + 1 == k) {
                result[i] = nums[queue.peekFirst()];
                i++;
            }
        }
        return result;
    }
}
```

Time O(n) | **Space** O(k)

Max Variable Window — Sum

#1004 Max Consecutive Ones III

Description: Flip at most k zeros. Return the length of the longest subarray of 1s.
State: count of zeros in window. Invalid when `zeros > k`.

Intuition: "Flip at most k zeros" = longest window containing at most k zeros; grow, and shrink only when zeros exceed k.

```
class Solution {
    public int longestOnes(int[] nums, int k) {
        int i = 0, zeros = 0, result = 0;
        for (int j = 0; j < nums.length; j++) {
            if (nums[j] == 0) zeros++;
            while (zeros > k) {
                if (nums[i++] == 0) zeros--;
            }
            result = Math.max(result, j - i + 1);
        }
        return result;
    }
}
```

Time O(n) | **Space** O(1)

Max Variable Window — Frequency Map

#3 Longest Substring Without Repeating Characters

Description: Find the length of the longest substring with all unique characters.
State: `freq[]` array. Invalid when `freq[nums[j]] > 1` after expanding.

Intuition: A repeat appears the moment you add it, so shrink from the left until that one character is unique again.

```
class Solution {
    public int lengthOfLongestSubstring(String s) {
        int[] freq = new int[128];
        int i = 0, result = 0;
        for (int j = 0; j < s.length(); j++) {
            freq[s.charAt(j)]++;
            while (freq[s.charAt(j)] > 1) freq[s.charAt(i++)]--;
            result = Math.max(result, j - i + 1);
        }
        return result;
    }
}
```

Time O(n) | **Space** O(1)

#424 Longest Repeating Character Replacement

Description: Replace at most k characters. Find the longest substring with one repeated char.
State: `freq[]` + `maxFreq`. Invalid when `(window size) - maxFreq > k` (too many to replace).
Intuition: A window is valid if the non-dominant chars (size – maxFreq) fit within k replacements; otherwise shrink.

Note: `maxFreq` never decreases when shrinking — we only care about windows larger than current best, so a stale maxFreq is safe and avoids a rescan.

```
class Solution {
    public int characterReplacement(String s, int k) {
        int[] freq = new int[26];
        int i = 0, maxFreq = 0, result = 0;
        for (int j = 0; j < s.length(); j++) {
            freq[s.charAt(j) - 'A']++;
            maxFreq = Math.max(maxFreq, freq[s.charAt(j) - 'A']);
            while (j - i + 1 - maxFreq > k) freq[s.charAt(i++) - 'A']--;
            result = Math.max(result, j - i + 1);
        }
        return result;
    }
}
```

Time O(n) | **Space** O(1)

Min Variable Window — Sum

#209 Minimum Size Subarray Sum

Description: Find the minimum length subarray with sum ≥ target.
State: running `sum`. Valid when `sum >= target` — record then shrink.

Intuition: Once the window reaches target, every extra left-trim that stays ≥ target gives a shorter candidate.

```
class Solution {
    public int minSubArrayLen(int target, int[] nums) {
        int i = 0, sum = 0, result = Integer.MAX_VALUE;
        for (int j = 0; j < nums.length; j++) {
            sum += nums[j];
            while (sum >= target) {
                result = Math.min(result, j - i + 1);
                sum -= nums[i++];
            }
        }
        return result == Integer.MAX_VALUE ? 0 : result;
    }
}
```

Time O(n) | **Space** O(1)

Min Variable Window — Frequency Map

#76 Minimum Window Substring

Description: Find the shortest substring of s that contains all characters of t.
State: `need[]` freq array + `required` counter. Valid when `required == 0`.

Intuition: Expand until all of t is covered, then shrink as far as you can while still covering it to find the tightest window.

```
class Solution {
    public String minWindow(String s, String t) {
        int[] need = new int[128];
        for (char c : t.toCharArray()) need[c]++;
        int required = t.length(), i = 0, start = 0, minLen = Integer.MAX_VALUE;
        for (int j = 0; j < s.length(); j++) {
            if (need[s.charAt(j)]-- > 0) required--;          // expand
            while (required == 0) {
                if (j - i + 1 < minLen) { minLen = j - i + 1; start = i; }
                if (need[s.charAt(i)]++ >= 0) required++;     // shrink
                i++;
            }
        }
        return minLen == Integer.MAX_VALUE ? "" : s.substring(start, start + minLen);
    }
}
```

Time O(n) | **Space** O(1)

Side-by-Side: 567 vs 76

Both use `need[]` + `required`. One difference:

```
                  567 (fixed, boolean)        76 (variable min, string)
Window size:      fixed = s1.length()         variable
Shrink trigger:   always when full            while required == 0
Record:           if required == 0 → true     minLen = min(minLen, j-i+1)
Return:           boolean                     s.substring(start, start+minLen)
```

Side-by-Side: 209 vs 76

Both are Min window. One uses sum, one uses map:

```
                  209 (sum)                 76 (map)
State:            int sum                   int[] need + int required
Valid condition:  sum >= target             required == 0
Expand:           sum += nums[j]            need[c]-- > 0 → required--
Shrink:           sum -= nums[i]            need[c]++ >= 0 → required++
```

Choosing the Template

Question asks for	Template	i moves
Fixed-size window operation	Fixed	when `j-i+1 == k`
Longest / maximum window	Max variable	while invalid (`while`)
Shortest / minimum window	Min variable	while valid (`while`)

Window content	State to maintain
Sum of values	`int sum`
Unique characters / no duplicates	`int[] freq` (128 or 26) or `Set`
Character frequency match	`int[] need` + `int required` counter
Running maximum	`Deque<Integer>` monotone decreasing indices

Fixed Window — Freq Array Compare

#438 Find All Anagrams in a String

Description: Given strings `s` and `p`, return a list of all start indices of `p`'s anagrams in `s`. (An anagram uses same characters with same frequencies.)

Intuition: An anagram is a fixed-length window whose letter counts equal p's, so slide width-`p.length()` and compare counts.

Algorithm: Fixed-size window of length `p.length()`. Maintain a frequency count for the window. Compare with `p`'s frequency count at each step using `Arrays.equals`.

```
class Solution {
    public List<Integer> findAnagrams(String s, String p) {
        List<Integer> result = new ArrayList<>();
        int[] need = new int[26], window = new int[26];
        for (char c : p.toCharArray()) need[c - 'a']++;
        int i = 0, j = 0;
        while (j < s.length()) {
            window[s.charAt(j++) - 'a']++;          // expand right
            if (j - i == p.length()) {              // ← VARIATION: fixed window size
                if (Arrays.equals(window, need)) result.add(i);
                window[s.charAt(i++) - 'a']--;      // shrink left
            }
        }
        return result;
    }
}
```

Time O(n) | **Space** O(1)

Max Variable Window — K-Distinct Frequency Map

#159 Longest Substring with At Most 2 Distinct Characters

Description: Return the length of the longest substring with at most 2 distinct characters.

Intuition: Grow the window freely; whenever a third distinct character appears, drop from the left until only 2 remain.

Algorithm: Max-variable sliding window. Expand `j`; shrink `i` when the frequency map has more than 2 distinct characters.

```
class Solution {
    public int lengthOfLongestSubstringTwoDistinct(String s) {
        Map<Character, Integer> freq = new HashMap<>();
        int i = 0, max = 0;
        for (int j = 0; j < s.length(); j++) {
            freq.merge(s.charAt(j), 1, Integer::sum);
            while (freq.size() > 2) {                  // ← VARIATION: shrink when >2 distinct
                char c = s.charAt(i++);
                freq.merge(c, -1, Integer::sum);
                if (freq.get(c) == 0) freq.remove(c);
            }
            max = Math.max(max, j - i + 1);
        }
        return max;
    }
}
```

Time O(n) | **Space** O(1)

#340 Longest Substring with At Most K Distinct Characters

Description: Return the length of the longest substring with at most `k` distinct characters.

Intuition: Same as #159 but the distinct cap is `k`; shrink whenever the map holds more than k distinct characters.

Variation: Parameterized generalization of #159 — replace the hard-coded `2` with `k`.

```
class Solution {
    public int lengthOfLongestSubstringKDistinct(String s, int k) {
        Map<Character, Integer> freq = new HashMap<>();
        int i = 0, max = 0;
        for (int j = 0; j < s.length(); j++) {
            freq.merge(s.charAt(j), 1, Integer::sum);
            while (freq.size() > k) {                  // ← VARIATION: parameterized k distinct
                char c = s.charAt(i++);
                freq.merge(c, -1, Integer::sum);
                if (freq.get(c) == 0) freq.remove(c);
            }
            max = Math.max(max, j - i + 1);
        }
        return max;
    }
}
```

Time O(n) | **Space** O(k)

Fixed Window — Sum vs Threshold (No Division)

#1343 Number of Sub-arrays of Size K and Average Greater than or Equal to Threshold

Description: Return the number of contiguous subarrays of size `k` whose average is greater than or equal to `threshold`.

Intuition: "Average ≥ threshold" over fixed width k is just "sum ≥ k·threshold"; slide a width-k window and count the hits.

Variation: fixed-size sliding window. To avoid floating-point division, compare `windowSum >= k * threshold`.

```
class Solution {
    public int numOfSubarrays(int[] arr, int k, int threshold) {
        int target = k * threshold;                 // compare sum vs k*threshold (no division)
        int windowSum = 0, count = 0;
        int i = 0;                                  // i = left edge (standard i/j window)
        for (int j = 0; j < arr.length; j++) {
            windowSum += arr[j];
            if (j - i + 1 == k) {                   // window has reached size k
                if (windowSum >= target) count++;
                windowSum -= arr[i++];              // slide: drop left, stay size k
            }
        }
        return count;
    }
}
```

Time O(n) | **Space** O(1)